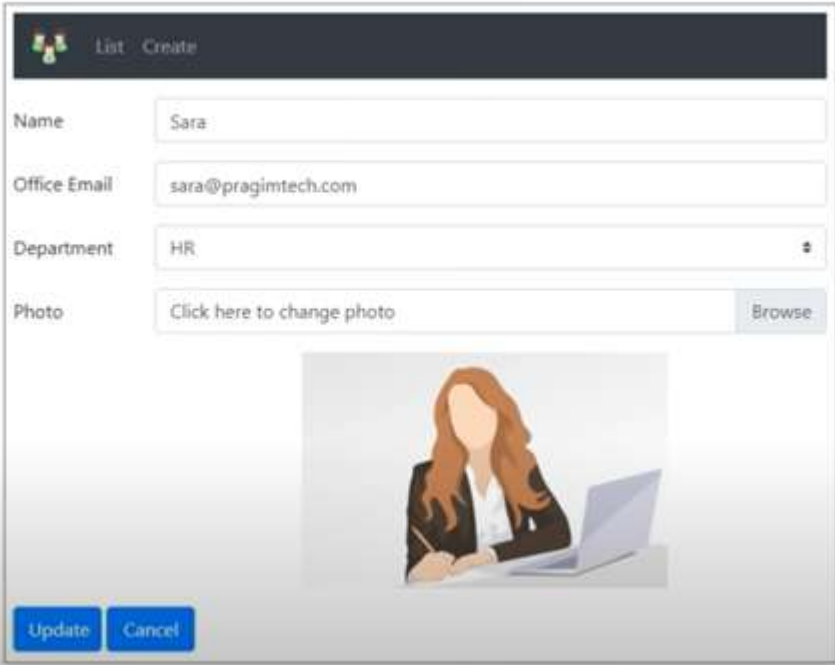


55 – [Edit View](#)

В този урок ще дадем възможност на потребителя да редактира съществуващи данни.

Edit View in ASP.NET Core MVC



Чрез бутона Update да бъдат изпратени до сървъра и презаписани върху съществуващите записи в БД, включително снимката чрез Edit view + Edit action.

Първото нещо е да направим активен бутон Edit, който да ни пренасочи към нов специален Edit action in HomeController.



Не може едновременно да се използват `href="#"` + `asp-action="edit"`

Редактираме по модела на другия бутон Edit, като уникалните данни на бутона се предават по `asp-route-id="@employee.Id"` където имаме локална foreach loop variable employee in Model.

```
Details.cshtml | Index.cshtml* - X
@model IEnumerable<Employee>

@{
    ViewBag.Title = "Employee List";
}

<div class="card-group">
    @foreach (var employee in Model)
    {
        var photoPath = "~/images/" + (employee.PhotoPath ?? "noimage.jpg");

        <div class="card m-3" style="min-width: 20rem; max-width: 30.5%;">
            <div class="card-header">
                <h3>@employee.Name</h3>
            </div>
            
            <div class="card-footer text-center">
                <a asp-controller="home" asp-action="details" asp-route-id="@employee.Id"
                class="btn btn-primary m-1">View</a>
                <a asp-controller="home" asp-action="edit" asp-route-id="@employee.Id"
                class="btn btn-primary m-1">Edit</a>
                <a href="#" class="btn btn-danger m-1">Delete</a>
            </div>
        </div>
    }
</div>
```

Друг Edit btn се намира в Details view, за който е необходим почти идентичен код, където обаче моделът е друг и трябва да се съобразим с него:

```
Details.cshtml | Index.cshtml*
@model HomeDetailsViewModel

@{
    ViewBag.Title = "Employee Details";
    var photoPath = "~/images/" + (Model.Employee.PhotoPath ?? "noimage.jpg");
}

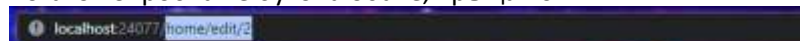
<div class="row justify-content-center m-1">
    <div class="col-sm-4">
        <div class="card">
            <div class="card-header">
                <h1>@Model.Employee.Name</h1>
            </div>

            <div class="card-body text-center">
                

                <h4>Employee ID: @Model.Employee.Id</h4>
                <h4>Employee Email: @Model.Employee.Email </h4>
                <h4>Employee Department: @Model.Employee.Department </h4>
            </div>

            <div class="card-footer text-center">
                <a asp-controller="home" asp-action="index" class="btn btn-primary">Back</a>
                <a asp-controller="home" asp-action="edit" asp-route-id="@Model.Employee.Id"
                class="btn btn-primary">Edit</a>
                <a href="#" class="btn btn-danger">Delete</a>
            </div>
        </div>
    </div>
</div>
```

Когато изпробваме бутона обаче, връща 404



This localhost page can't be found

No webpage was found for the web address: <http://localhost:24077/home/edit/2>

HTTP ERROR 404

защото още в HomeController все още нямаме Edit action, който ще бъде много сходен с Create:

```

HomeController.cs
EmployeeManagement.Controllers.HomeController

[HttpGet]
0 references
public ActionResult Create()
{
    return View();
}

[HttpPost]
0 references
public IActionResult Create(EmployeeCreateViewModel model)
{

```

Трябва да може да поеме и прехвърли всички данни, затова създаваме EmployeeEditViewModel.cs class

Понеже за редакцията на формата ни трябва много данни, които съвпадат с EmployeeCreateViewModel,

```

public class EmployeeCreateViewModel
{
    [Required]
    [MaxLength(50, ErrorMessage = "Name cannot exceed 50 characters")]
    1 reference
    public string Name { get; set; }
    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_\.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-]+\.$",
        ErrorMessage = "Invalid email format")]
    [Display(Name = "Office Email")]
    1 reference
    public string Email { get; set; }
    [Required]
    1 reference
    public Dept? Department { get; set; }

    3 references
    public IFormFile Photo { get; set; }
}

```

вместо да ги Copy->Paste, най-разумното решение е, те да бъдат наследени, като се извикат. По този начин наследява Name, Email, Department, Photo, но ни трябва и Id и съществуващ PhotoPath:

```

0 references
public class EmployeeEditViewModel : EmployeeCreateViewModel
{
    0 references
    prop public int Id { get; set; }
    0 references
    public string ExistingPhotoPath { get; set; }
}

```

Декларирахме необходимите ни property – свойства, но сега ни трябва данните от БД:


```

HomeController.cs* EmployeeCreateViewModel.cs Details.cshtml Index.cshtml
EmployeeManagement.Controllers.HomeController Create()

0 references
public HomeController(IEmployeeRepository employeeRepository,
                     IWebHostEnvironment hostingEnvironment)
{
    _employeeRepository = employeeRepository;
    this.hostingEnvironment = hostingEnvironment;
}
0 references
public IActionResult Index()

```

По-надолу вече са actions:

Най-просто е да декларираме id, което автоматично ще бъде прихванато от URL заради MVC, след това създаваме нова локална променлива, тип Employee, която да извлече от БД свойствата по id, а после да ги предаде на създадената за целта инстанция.

```

HomeController.cs* EmployeeCreateViewModel.cs Details.cshtml Index.cshtml
EmployeeManagement.Controllers.HomeController Create()

[HttpGet]
0 references
public IActionResult Create()
{
    return View();
}

[HttpGet]
0 references
public IActionResult Edit(int id)
{
    new local var Employee employee = _employeeRepository.GetEmployee(id);
    new instance EmployeeEditViewModel employeeEditViewModel = new EmployeeEditViewModel
    {
        new = local, DB
        Id = employee.Id,
        Name = employee.Name,
        Email = employee.Email,
        Department = employee.Department,
        ExistingPhotoPath = employee.PhotoPath
    };
    return View(employeeEditViewModel);
}

[HttpPost]
0 references
public IActionResult Create(EmployeeCreateViewModel model)

```

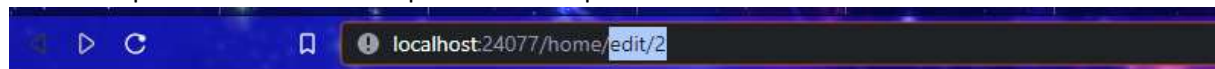
```

[HttpGet]
public IActionResult Edit(int id)
{
    Employee employee = _employeeRepository.GetEmployee(id);
    EmployeeEditViewModel employeeEditViewModel = new
EmployeeEditViewModel
{
    Id = employee.Id,
    Name = employee.Name,
    Email = employee.Email,
    Department = employee.Department,
    ExistingPhotoPath = employee.PhotoPath

```

```
};
return View(employeeEditViewModel);
}
```

Този път грешката е позната и приятна за отстраняване

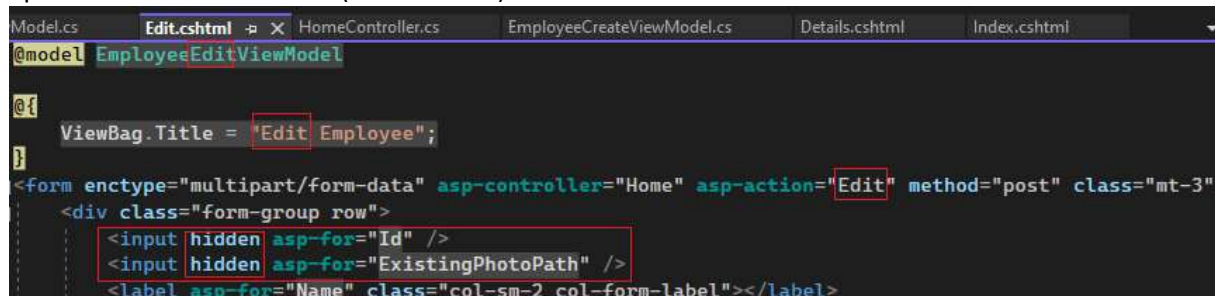


An unhandled exception occurred while processing the request.

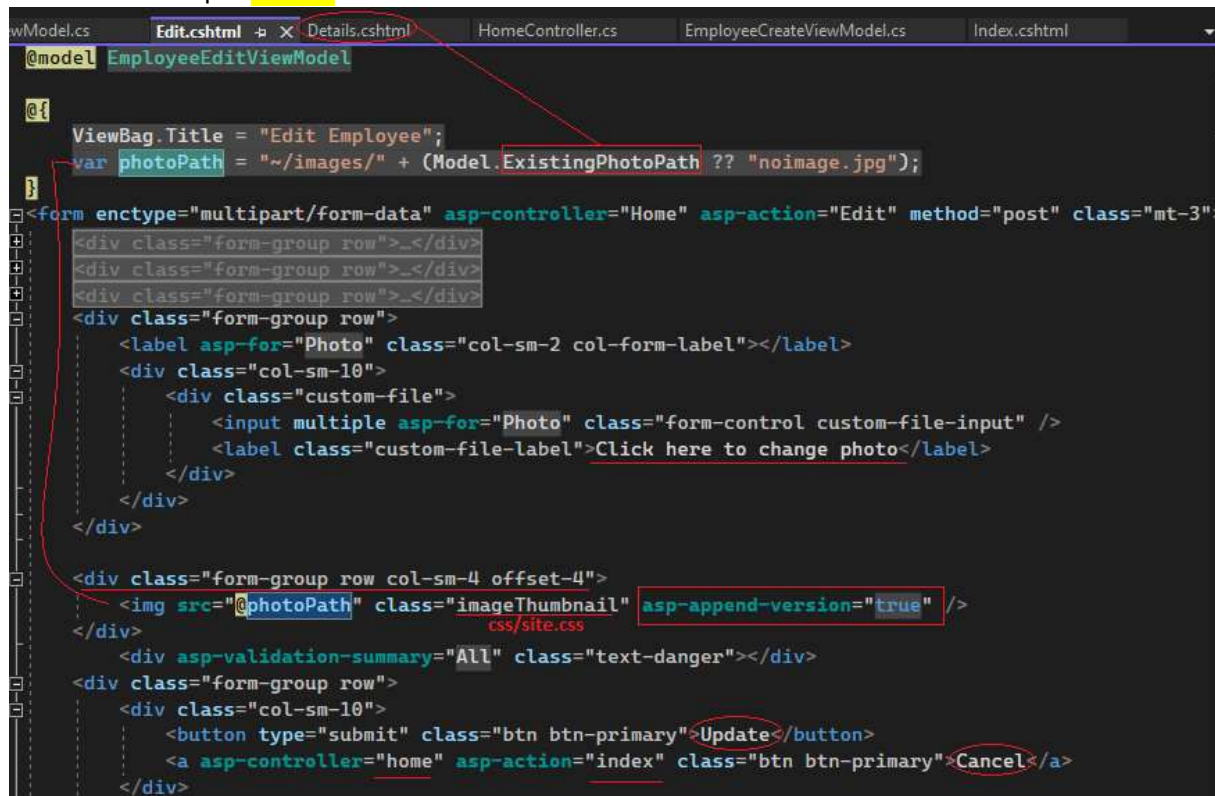
InvalidOperationException: The view 'Edit' was not found. The following locations were searched:
 /Views/Home/Edit.cshtml
 /Views/Shared/Edit.cshtml
 /Pages/Shared/Edit.cshtml

Microsoft.AspNetCore.Mvc.ViewEngines.ViewEngineResult.EnsureSuccessful(IEnumerable<string> originalLocations)

Трябва да създадем View->Home->Edit.cshtml, който е много сходен с Create, който ще копираме за леки изменения (Create->Edit):



Искаме данните за Id, ExistingPhotoPath да бъдат прихванати, но да не се показват, затова ги поставяме в .



Данните са автоматично прихванати.

localhost:24077/home/edit/2



List Create

Name

John

Office Email

john@abv.bg

Department

HR

Photo

Choose Files

No file chosen

Click here to change photo



Update

Cancel

На този етап при клик Edit виждаме попълнените данни на потребителите и имаме възможност за избор на картинка, при Cancel се връщаме в Index, но при натискане на бутон Update връща 404, защото все още нямаме POST Edit в HomeController.

1) рамкиране на свойства - properties (ViewModel)

Step 1 : Create Edit View Model

```
public class EmployeeEditViewModel : EmployeeCreateViewModel
{
    public int Id { get; set; }
    public string ExistingPhotoPath { get; set; }
}

public class EmployeeCreateViewModel
{
    [Required]
    [MaxLength(50, ErrorMessage = "Name cannot exceed 50 characters")]
    public string Name { get; set; }
    [Required]
    [RegularExpression(@"^[a-zA-Z0-9_+]+\@[a-zA-Z0-9]+\.[a-zA-Z0-9-]+\.$",
        ErrorMessage = "Invalid Email Format")]
    [Display(Name = "Office Email")]
    public string Email { get; set; }
    [Required]
    public Dept? Department { get; set; }
    public IFormFile Photo { get; set; }
}
```

2) прихващане и обработка на данни (HomeController)

Step 2 : Implement Edit Action that responds to HttpGet request

```
[HttpGet]
public ActionResult Edit(int id)
{
    Employee employee = _employeeRepository.GetEmployee(id);
    EmployeeEditViewModel employeeEditViewModel = new EmployeeEditViewModel
    {
        Id = employee.Id,
        Name = employee.Name,
        Email = employee.Email,
        Department = employee.Department,
        ExistingPhotoPath = employee.PhotoPath
    };
    return View(employeeEditViewModel);
}
```

3) показване/дисплей .cshtml (View)

Step 3 : Implement Edit View

```
@model EmployeeEditViewModel

@{
    ViewBag.Title = "Edit Employee";
    var photoPath = "~/images/" + (Model.ExistingPhotoPath ?? "noimage.jpg");
}

<form asp-controller="home" asp-action="edit" enctype="multipart/form-data" method="post">

    <div asp-validation-summary="All" class="text-danger"></div>

    <input hidden asp-for="Id" />
    <input hidden asp-for="ExistingPhotoPath" />

    Please refer to our blog at the following link for complete view code
    http://csharp-video-tutorials.blogspot.com/

</form>
```